

LABORATORIO DI PROGRAMMAZIONE I

Lezione 8
Liste e List comprehension

Laura Nenzi

List comprehension

Usando il ciclo for:

```
quadrati = []  
for n in range(10):  
    quadrati.append(n**2)  
print(quadrati)  
#[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

Usando la list comprehension:

```
quadrati = [n**2 for n in range(10)]  
print(quadrati)  
#[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

List comprehension

È una sintassi che permette di creare liste nuove applicando una trasformazione o un filtro a un iterabile esistente (come una lista, una tupla o un range):

$[f(x) \text{ for } x \text{ in } X]$

L'operazione che
vuoi applicare

Il ciclo che itera sugli elementi
dell'iterabile X.

Il risultato quindi per $X = \{x_1, \dots, x_n\}$ è $[f(x_1), \dots, f(x_n)]$

List comprehension

Usando il ciclo for:

```
pari = []  
for n in range(10):  
    if n%2==0:  
        pari.append(n)  
print(pari)
```

Usando la list comprehension:

```
pari2 = [n for n in range(10) if n%2==0]  
print(pari2)
```

List comprehension

È una sintassi che permette di creare liste nuove applicando una trasformazione o un filtro a un iterabile esistente (come una lista, una tupla o un range):

[espressione for elemento in iterabile if condizione]

L'operazione che vuoi applicare

Il ciclo che itera sugli elementi dell'iterabile.

(opzionale): Un filtro per includere solo gli elementi che soddisfano una certa condizione.

List comprehension

Usando il ciclo for:

```
lista_uno = [1, 2, 3]
lista_due = [3, 1, 4]
mix = []
# annidamento di cicli for
for x in lista_uno:
    for y in lista_due:
        if x != y:
            mix.append((x, y))
print(mix) # [(1, 3), (1, 4), (2, 3), (2, 1), (2, 4), (3, 1), (3, 4)]
```

Usando la list comprehension:

```
mix = [(x, y) for x in lista_uno for y in lista_due if x != y]
print(mix) # [(1, 3), (1, 4), (2, 3), (2, 1), (2, 4), (3, 1), (3, 4)]
```

Set/dictionary comprehension

Non è limitato solo alla **list comprehension**:

- **set comprehension**

- E.g.: `{n**2 for n in [1,2,2,2,3,4,5]}`

- **dictionary comprehension**

- E.g.: `{n: n**2 for n in [1,2,2,2,3,4,5]}`

Iteratori

- Una collezione iterabile è un modo di rappresentare gli oggetti secondo una qualche struttura, definendo il concetto di **cammino sugli stessi**. E.g.:
 - 1,2,3,...10
 - "b","a","n","a","n","a"
- In Python un **iteratore** è un oggetto su cui è possibile iterare, ovvero un oggetto che ha una nozione implicita di iterazione.

Iteratori

- Il meccanismo di iterazione ritorna un elemento per volta e viene realizzato implementando necessariamente due magic methods:
 - `__iter__`, che inizializza l'iteratore e lo ritorna tramite `self` (perchè l'iteratore essendo implicito fa parte dell'oggetto stesso, quindi è `self`)
 - `__next__`, che ritorna il prossimo valore dell'iteratore da visitare (o genera una eccezione `StopIteration` se non c'è un valore successivo)

Iteratori

```
mystr = "banana"
myiter = iter(mystr) # costruisco iteratore

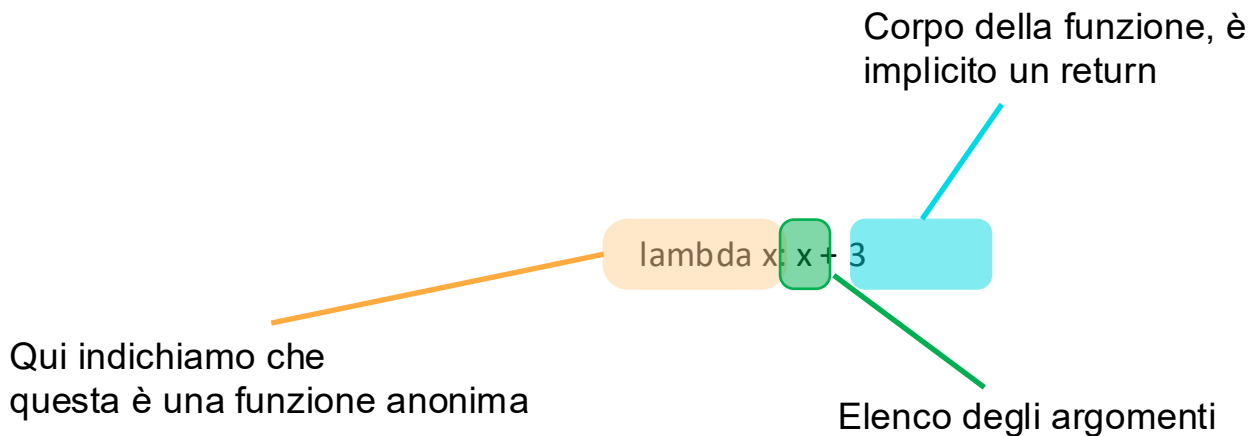
print(next(myiter)) # b
print(next(myiter)) # a
print(next(myiter)) # n
print(next(myiter)) # a
print(next(myiter)) # n
print(next(myiter)) # a
```

costruisco l'iteratore con la chiamata `iter`, usando il fatto che qualcuno ha implementato un magic method `__iter__` per le stringhe in Python

Le chiamate che restituiscono i valori su cui si muove l'iteratore sono tutte della forma `next(myiter)`

Funzioni anonime / lambda function

- Possono avere un numero qualsiasi di argomenti ma **una sola espressione**.
- L'espressione viene valutata e restituita.



Funzioni anonime/ lambda function

```
def f(x):  
    return x + 1
```

```
somma = lambda a, b: a + b  
print(somma(2, 3)) # 5
```

```
f = lambda x: x + 1  
print(f(5)) # 6
```

```
somma = lambda a, b: a + b  
print(somma(2, 3)) # 5
```

- Sono utili soprattutto come argomento di altre funzioni

Map, filter, reduce

- Molte operazioni su collezioni sono riconducibili a particolari applicazioni di funzioni su di esse:
 - **map(f, collection)**
applica la funzione f a tutti gli elementi della collezione
 - **filter(predicate, collection)**
applica il predicato (funzione che ritorna valori Booleani) agli elementi della collezione e mantiene solo quelli per cui il predicato è vero

Mappare:

applicare una funzione su ciascun elemento di una sequenza.

- Esempio: funzione che prende una lista di numeri interi in [0, 9] e ritorna una lista di stringhe, corrispondenti ai numeri scritti in Italiano

```
lista_numeri = [1,3,6]
dizionario_numeri_stringhe = {0: "zero", 1: "uno", 2: "due", 3: "tre", 4:
"quattro", 5: "cinque", 6: "sei", 7: "sette", 8: "otto", 9: "nove"}

list_comp =[dizionario_numeri_stringhe[n] for n in lista_numeri]

list_con_map = list(map(lambda n: dizionario_numeri_stringhe[n], lista_numeri))
print(list_comp, list_con_map)
```

Filtrare:

selezionare alcuni elementi di una lista per formare una sottolista

- Esempio: funzione che prende una lista di numeri e restituisce solo quelli pari

```
lista =[1,2,3,4,5,6,7,8,9]
list_comp = [n for n in lista if n%2==0]

list_con_filter = list(filter(n%2==0, lista))

print(list_comp, list_con_filter)
```

Map, filter, reduce

- **reduce(f, collection)**

usa la funzione f (che ha due argomenti) per ridurre la collezione ad un solo valore:

- Se f è *lambda* $x, y: x + y$ e la collezione $[1, 2, 3, 4]$ allora:
 $f(f(f(1, 2), 3), 4)$ corrisponde a $((1 + 2) + 3) + 4$

- Per usarlo in Python serve importarlo da *functools*

- Notate come questi risultati siano anche ottenibili usando list/dict comprehension

Ridurre: compattare una sequenza di elementi in un singolo valore

- Esempio di come fare il prodotto di tutti i numeri in una lista:

```
from functools import reduce

product = reduce((lambda x, y: x * y), [1, 2, 3, 4])
print(product)
```

Moduli della Libreria Standard di Python

- Abbiamo già visto che un modulo è un file che contiene un insieme di funzioni che vuoi includere.
- Considera un modulo come una libreria di codice
- Python dispone di una **varietà di moduli standard** che forniscono funzionalità utili senza bisogno di installazioni aggiuntive
- Moduli più comuni:
 - - **math** → Funzioni matematiche (trigonometria, logaritmi, ecc.).
 - - **random** → Generazione di numeri casuali.
 - - **statistics** → Funzioni statistiche.
 - - **datetime** → Classi per la gestione di date e orari.
 - - **csv** → Classi per leggere e scrivere dati in formato CSV.
 - - **re** → Supporto per espressioni regolari per la manipolazione e la ricerca di pattern.

Moduli della Libreria Standard di Python

- **from Statement** è utilizzato per **importare elementi specifici** da un modulo o libreria.
- Permette di importare **funzioni, classi o variabili specifiche** senza dover richiamare tutto il modulo.

```
import random  
  
random.randint(0,5)
```

```
from random import randint  
  
randint(0,10)
```

Librerie

- Una libreria è una raccolta di pacchetti o singoli moduli che vengono raggruppati per estendere le funzionalità.
- Tipologie:
 1. Librerie standard (incluse in Python)
 2. Librerie e pacchetti di terze parti (devono essere installati)
- Come per i moduli, utilizziamo le parole chiave `from` e `import`.
- Esempi di libreria standard di python è `os`

```
import os
```

```
# 1. Ottenere il percorso della cartella corrente  
current_dir = os.getcwd()  
print("Cartella corrente:", current_dir)
```

Libreria di terze parti

- Le libreria di terze parti devono essere installate
- Il metodo di installazione dipende dal **gestore di pacchetti** utilizzato.
- Esistono due modi principali per installarli:
 1. **Pip**- Se usi `pip` per la gestione dei pacchetti
 2. **Conda** - Se usi `conda` o `mamba` per la gestione dei pacchetti (è quello che utilizziamo noi)
- Per installare una libreria si fa da terminale con:

```
conda install matplotlib
```

```
pip install matplotlib
```

Libreria di terze parti

- Le librerie di terze parti comunemente utilizzate nella data sono:
 - **Pandas** → Strutture dati e operazioni per manipolare tabelle numeriche e serie temporali.
 - **NumPy** → Supporto per array e matrici multi-dimensionali, con funzioni matematiche avanzate.
 - **Matplotlib** → Libreria per creare grafici statici, animati e interattivi in Python.
 - **Seaborn** → Interfaccia avanzata per visualizzazioni statistiche e grafiche informative.
 - **SciPy** → Strumenti per calcoli scientifici e tecnici, tra cui ottimizzazione e algebra lineare.
 - **Scikit-learn** → Implementazione di algoritmi di machine learning per analisi e data mining.
 - **Plotly** → Creazione di grafici interattivi e visivamente accattivanti per il web.
- È comune assegnare un alias alla libreria, cioè un nome più breve

```
import matplotlib.pyplot as plt
```



alias

Esempio di grafico con matplotlib

```
import matplotlib.pyplot as plt
import random

y = [random.randint(1, 100) for _ in range(10)]

# Plot dei dati
plt.figure(figsize=(12, 6))
plt.plot(y, marker='o', linestyle='-', label='ettichetta')
plt.title('Grafico')
plt.xlabel('x')
plt.ylabel('y')
plt.legend()
plt.show()
```

